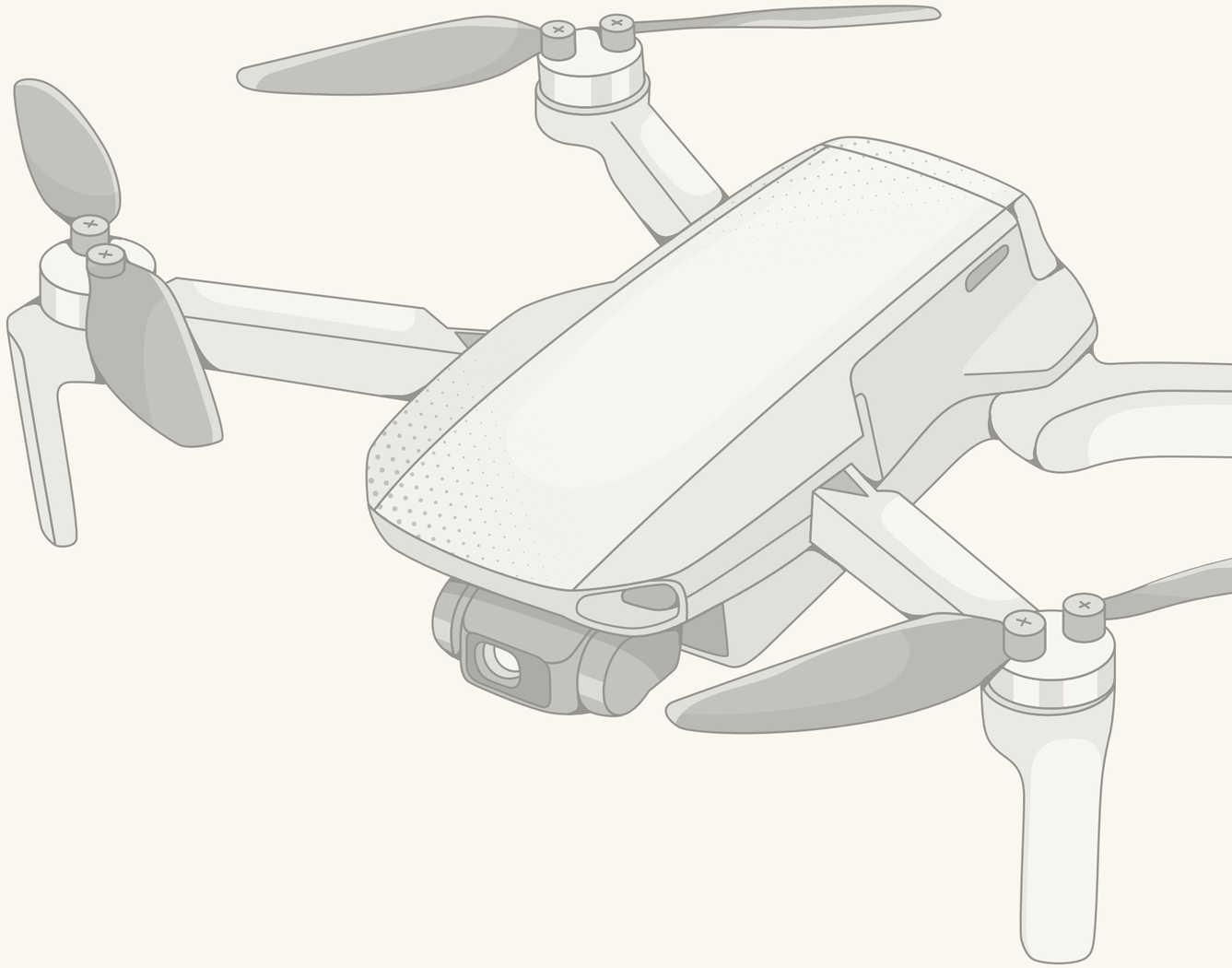
RRC SUMMER SCHOOL

Motion Planning - II



MATHAI MATHEW

BASICS

What you have learned so far...

We use senors like the depth camera and LIDAR to gain information from the environment and to make a map from that using methods like mapping.

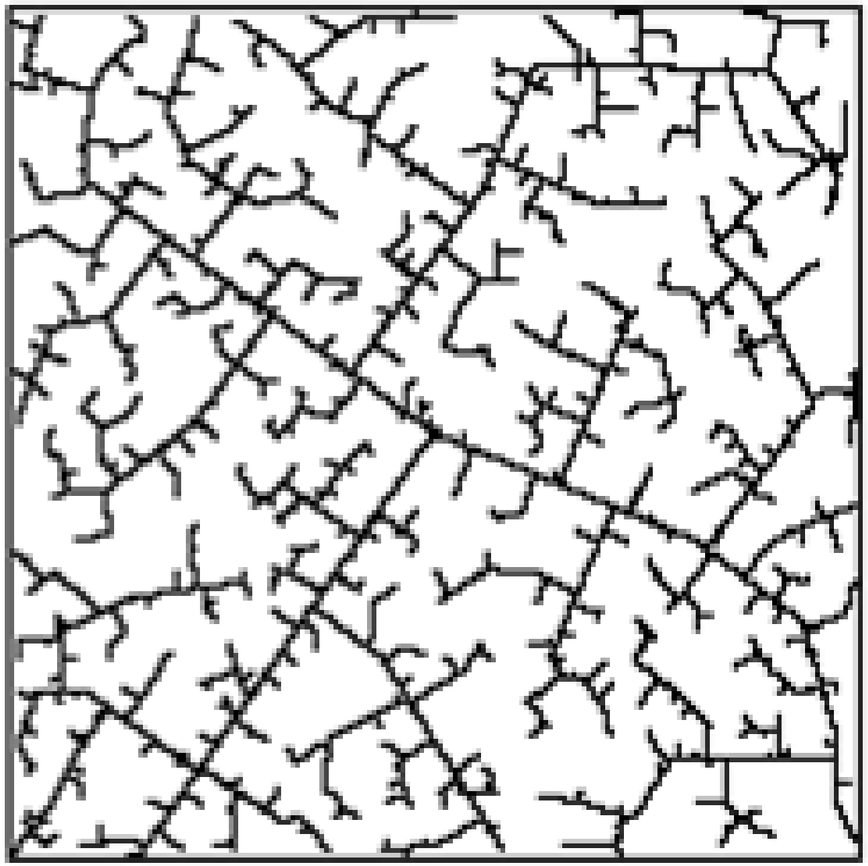
We then can use localization methods like AMCL and use odometry and sensor data to then get our location on the occupancy map.

We can then use the costmap and generate a path from our position to the goal using an algorithm like A* or RRT.

BASICS

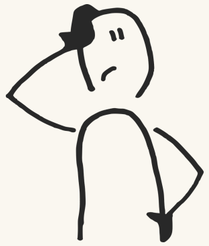
What’s wrong here?

45 iterations



390 iterations

The Gap Between a Path and Motion



Problem

- The path has **no timing**
- The path **ignores dynamics**
- **The path assumes a static world**

Solution

Using methods that incorporate these features so as to bridge the gap between our ideal scenario and reality.

TRAJECTORIES

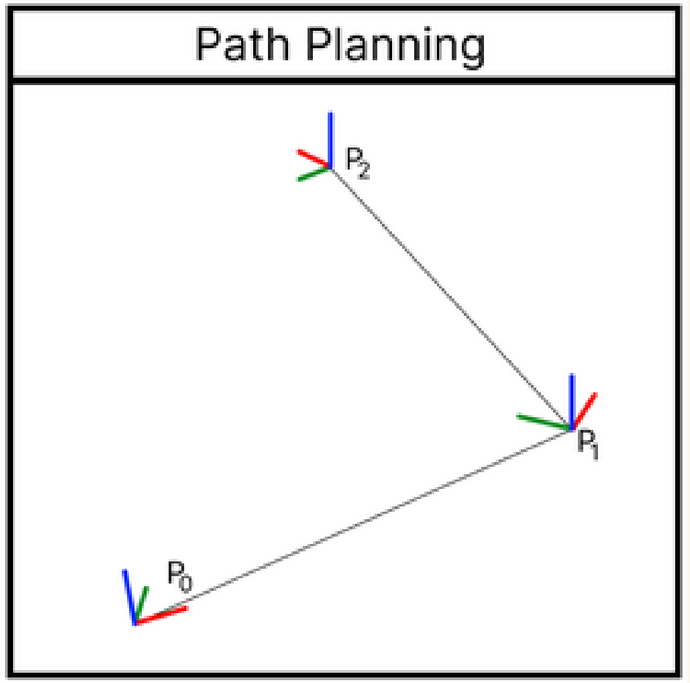
What is a Trajectory?

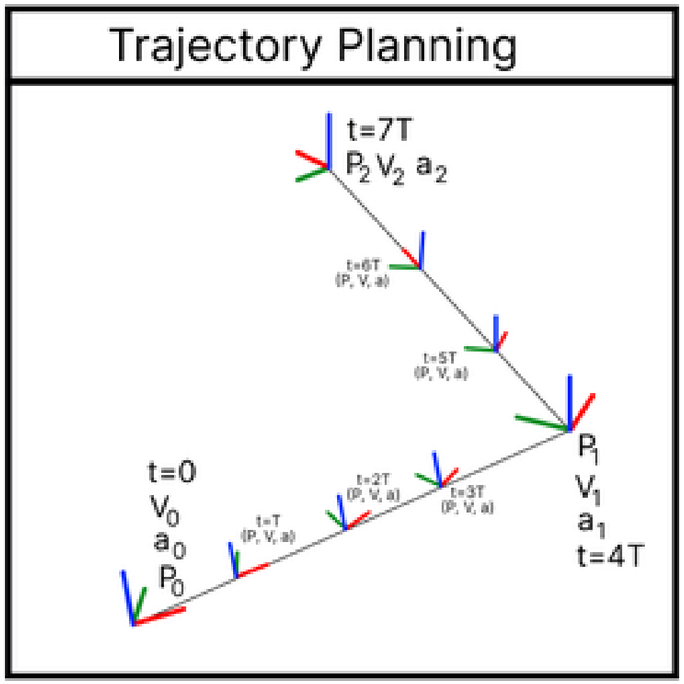
What is a Path

A path is simply a list of point that take us from point A to point B. This is chosen using the costmap using global planners.

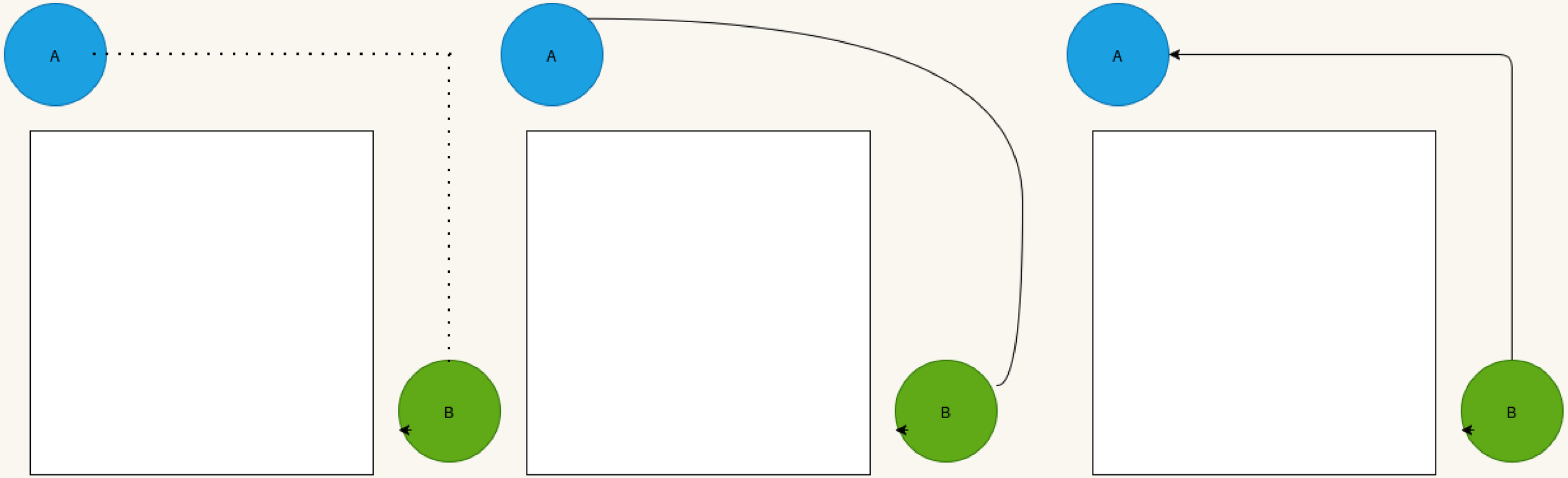
What about a trajectory?

- A trajectory in it’s most basic form, a path with additional data such as time, velocity, etc.
- This allows the trajectory to essentially say how to go from point A to B.
- **A path is a shape, a trajectory is a plan**





Lets look at some Paths.



Which Polynomial do we use?

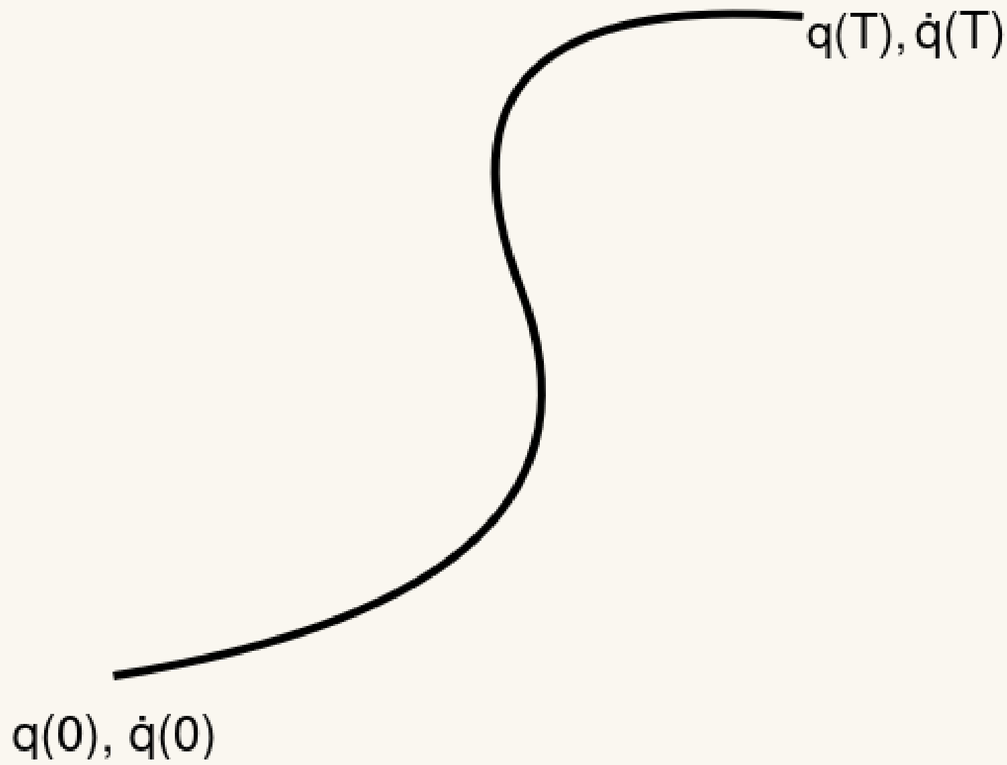
The cubic polynomial

We can use a polynomial of the form:

$$q(t) = a_0 + a_1\, t + a_2\, t^2 + a_3\, t^3$$

q(0) = start position
q(T) = end position
q̇(0) = start velocity
q̇(T) = end velocity

4 known values and 4 unknowns to solve for.
But what if we have many obstacles, not just a smooth path?



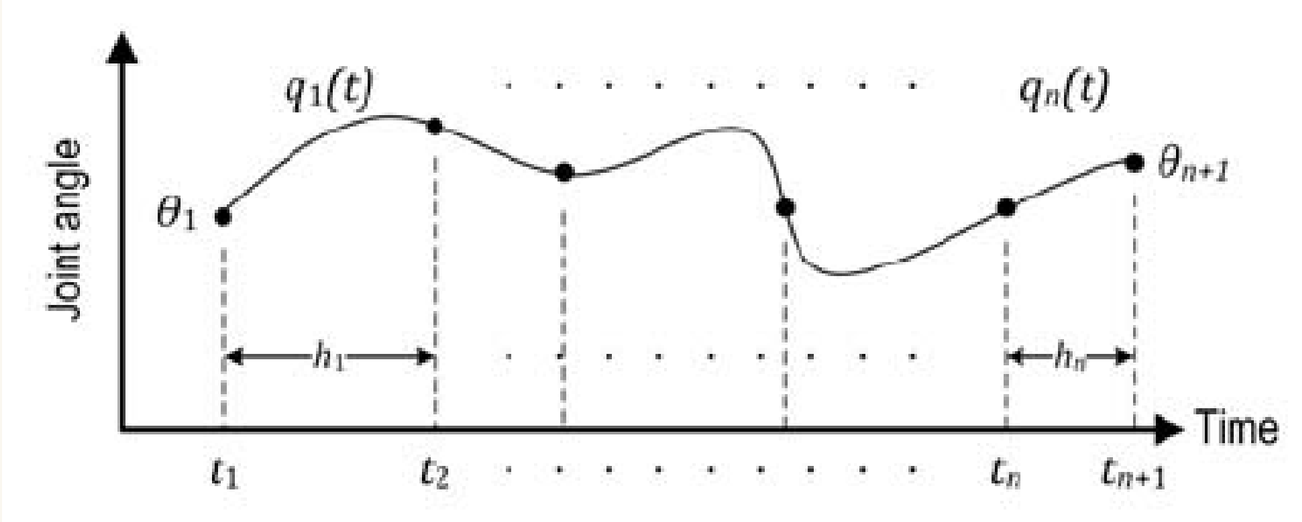
Which Polynomial do we use?

The piecewise polynomial

We can use a group of polynomials, all hatched at their end points.

This is a valid solution, but it has a few issues

- How do we decide on what velocities to chose at the intersection points?
- The calculations for all the constraints become more costly

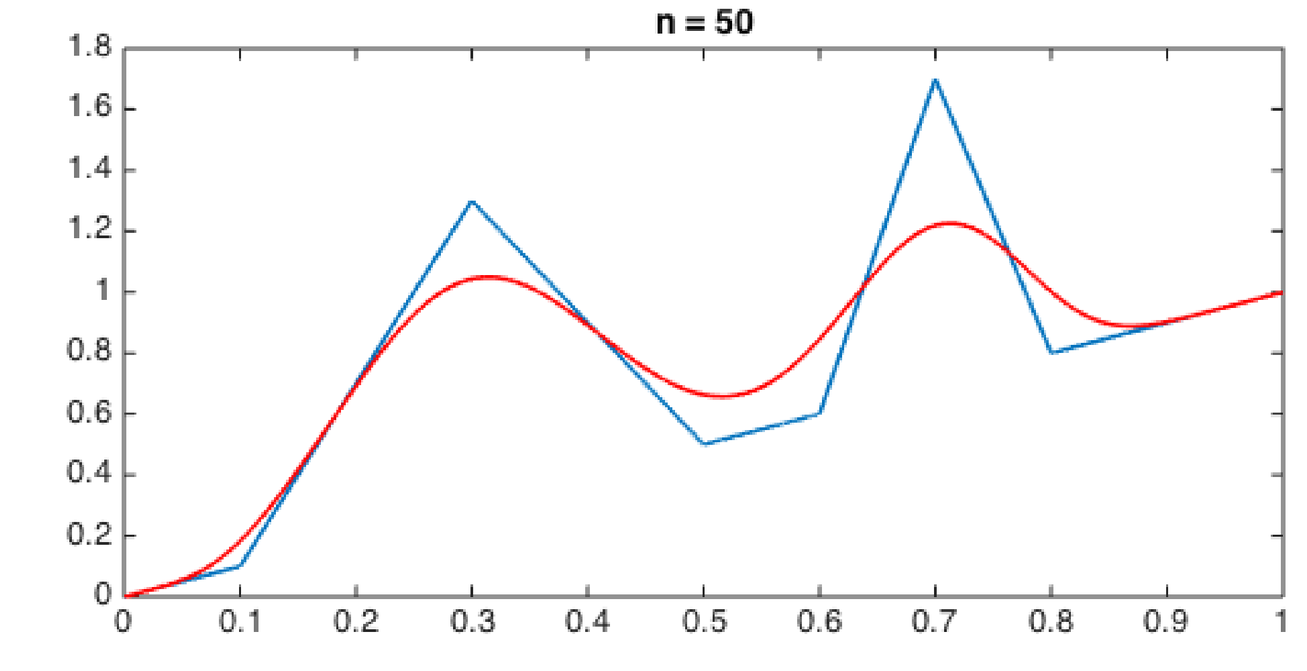


Bézier and Bernstein

To solve the previous problems we can use a Bernstein polynomial

$$\mathbf{B}(t) = \sum_{k=0}^{n} \binom{n}{k} (1-t)^{n-k} t^k \mathbf{P}_k, \quad t \in [0,1]$$

$$\dot{\mathbf{B}}(t) = n \sum_{k=0}^{n-1} \binom{n-1}{k} (1-t)^{n-1-k} t^k \left(\mathbf{P}k + 1 - \mathbf{P}k\right)$$



Which Path is Best?

We can find out using optimization

If all these paths are valid, what do we choose to follow?

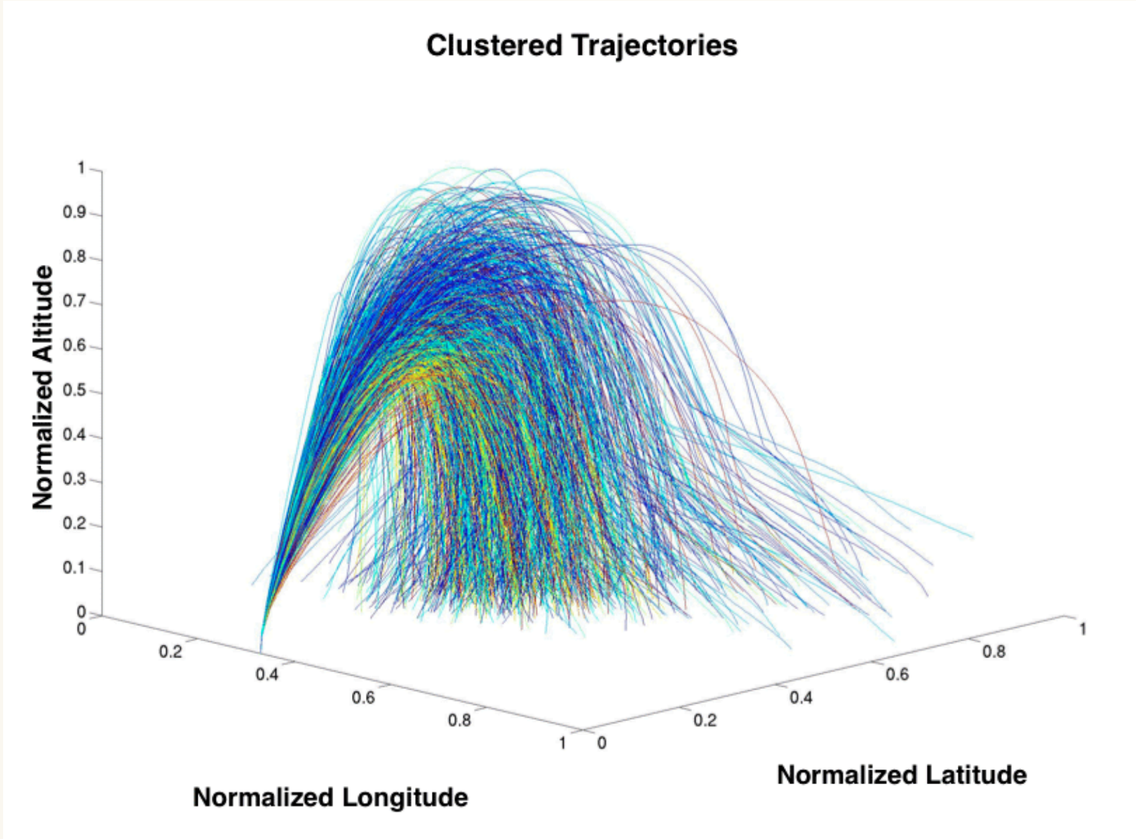
$$\min_{q(t)} \quad J(q(t))$$

$$|\ddot{q}(t)| \leq v_{\max}$$

$$|\ddot{q}(t)| \leq a_{\max}$$

$$q(t) \notin \mathcal{O} \quad \forall t \in [0, T]$$

$$q(0) = q_{\text{start}}, \quad q(T) = q_{\text{goal}}$$



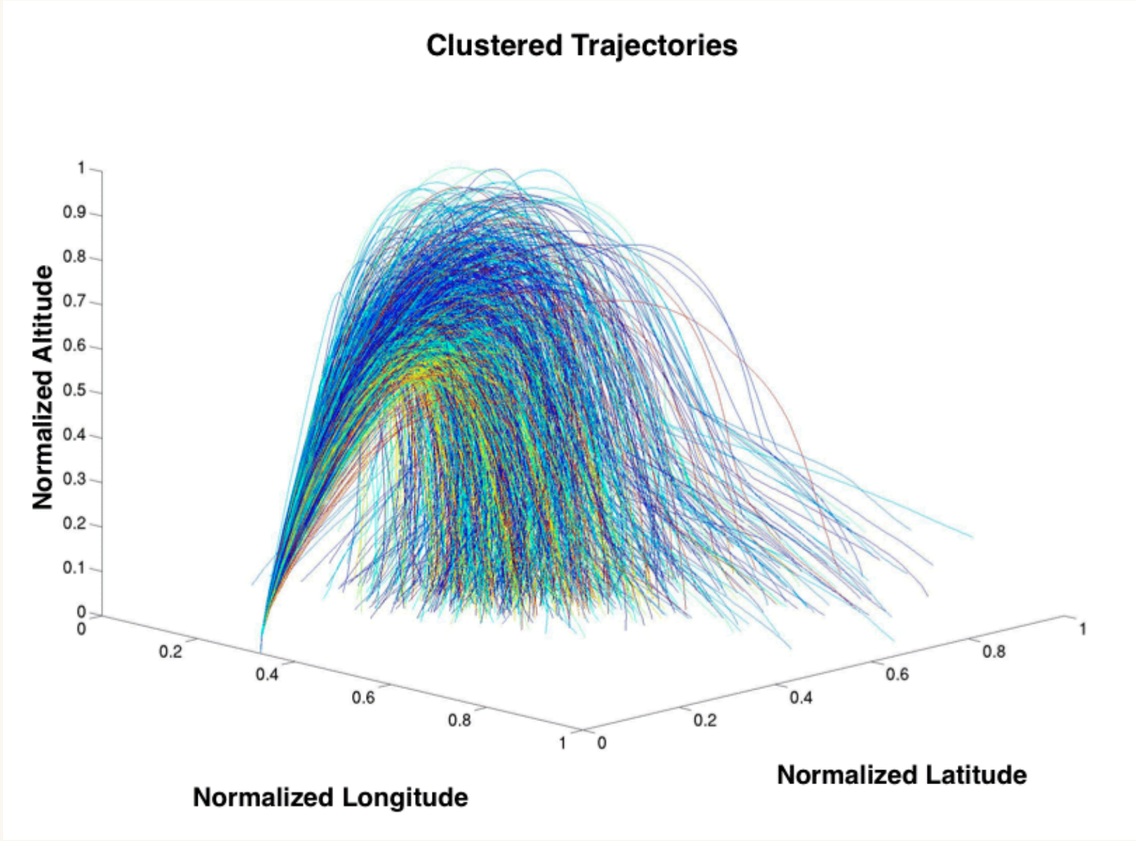
OPTIMIZATION

Which Path is Best?

We can find out using optimization

$$\min_{q(t)} \quad J(q(t))$$

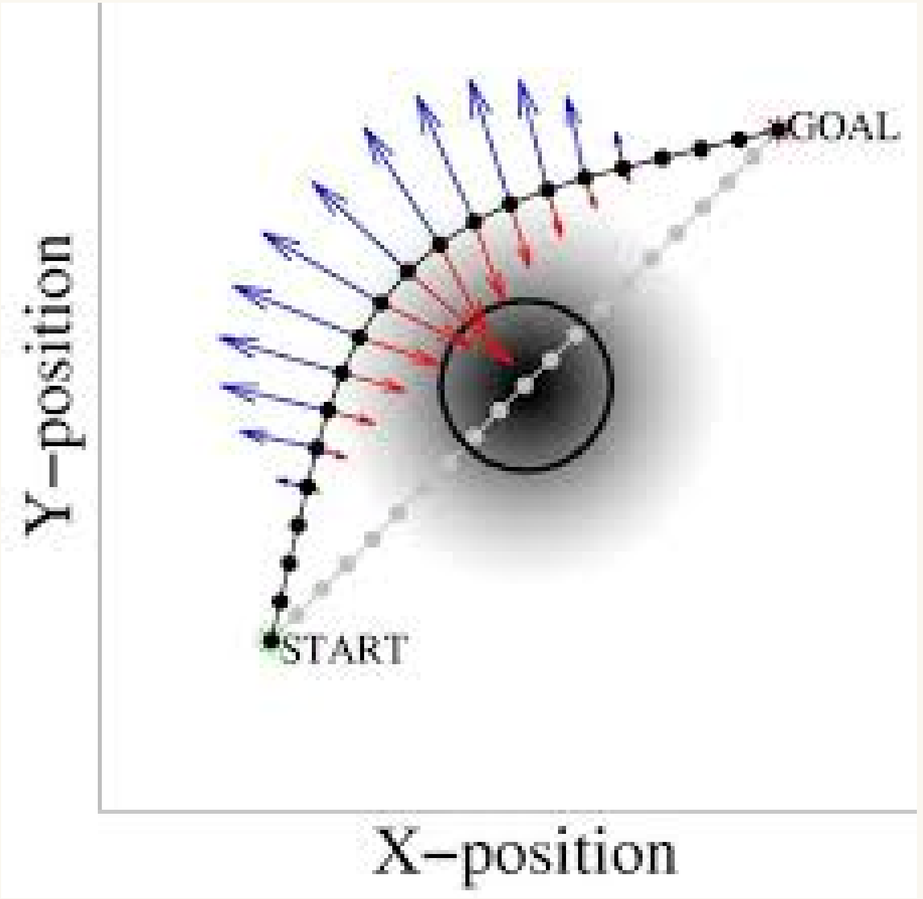
$$\min_{q(t)} \quad w_1 \int_0^T |\dddot{q}|^2 dt + w_2 T + w_3 \int_0^T \frac{1}{d(q(t))^2} dt$$



CHOMP

How does CHOMP work

CHOMP is an example of a more sophisticated optimization based planner, it calculates the gradient of the trajectory, and moves away from it, thus minimizing the cost function. It is widely used for manipulators

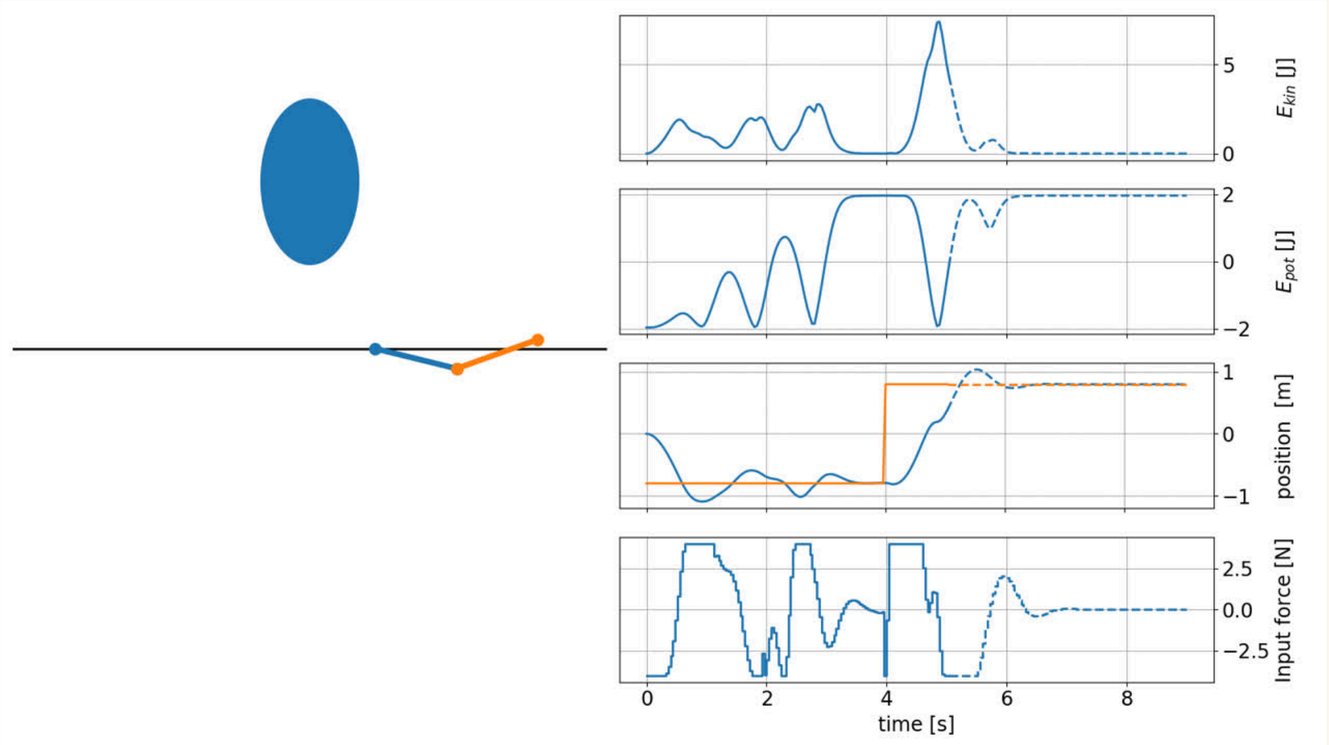


Model Predictive Control

The Receding Horizon for Planning

The main component of MPC that is different from other planners is that we use the dynamics of the robot to plan forward in time.

Plan N steps into the future and perform the best first step.



The Robot Model and Linearization

The unicycle model

State vector and control input:

$$\mathbf{x} = [x\ y\ \theta], \quad \mathbf{u} = [v\ \omega]$$

Dynamics:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = [v\cos\theta\ v\sin\theta\ \omega]$$

Note: this problem is non-linear due to the presence of sin and cos here.

Linearization

$$(\mathbf{x}\text{ref}, \mathbf{u}\text{ref})$$

$$\delta\mathbf{x} = \mathbf{x} - \mathbf{x}\text{ref}, \quad \delta\mathbf{u} = \mathbf{u} - \mathbf{u}\text{ref}$$

$$\delta\dot{\mathbf{x}} = A, \delta\mathbf{x} + B, \delta\mathbf{u}$$

MPC

The Full Problem

Cost Formulation

$$\min_{\mathbf{u}0,\ldots,\mathbf{u}N-1} \sum_{k=0}^{N-1} \left(\mathbf{x}_k^T Q, \mathbf{x}_k + \mathbf{u}_k^T R, \mathbf{u}_k\right) + \mathbf{x}_N^T Q_f, \mathbf{x}_N$$

subject to:

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k \quad \forall k \in [0, N-1]$$

$$v_{\min} \leq v_k \leq v_{\max}$$

$$\omega_{\min} \leq \omega_k \leq \omega_{\max}$$

$$\mathbf{x}_0 = \mathbf{x}_{\text{current}}$$

Title

```
Cgiven: current state x, goal x_goal, model A, B
given: cost matrices Q, R, Q_f
given: horizon N, timestep dt

loop:
    # 1. observe
    x_current = get_robot_state()

    # 2. relinearize model around current state
    A, B = linearize(f, x_current)

    # 3. build and solve the QP
    cost = 0
    for k in range(N):
        cost += (x_k - x_goal).T @ Q @ (x_k - x_goal)
        cost += u_k.T @ R @ u_k
    cost += (x_N - x_goal).T @ Q_f @ (x_N - x_goal)

    u_sequence = solve_QP(cost, constraints, x_current)

    # 4. apply only the first control input
    u_apply = u_sequence[0]
    send_to_robot(u_apply)

    # 5. wait and repeat
    wait(dt)
```

The End